ASSIGNMENT 15

Name: Rajdeep Das

Roll No: CSB25203

Subject: Advanced Programming

Program:

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

long long counter = 0;

/* Mutex variable */
pthread_mutex_t lock;

/* ---------- WITHOUT MUTEX ---------- */

void* increment_without_mutex(void* arg)
{
   int i;

   for(i = 0; i < INCREMENTS; i++)
   {
      /* Race condition occurs here */
      counter++;
   }
```

```
    return NULL;
}

/* ---------------- WITH MUTEX ---------------- */

void* increment_with_mutex(void* arg)
{
    int i;

    for(i = 0; i < INCREMENTS; i++)
    {
        /* Lock critical section */
        pthread_mutex_lock(&lock);

        counter++;

        /* Unlock critical section */
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int i;

    /* ================================================= */
    /* PART 1 : WITHOUT MUTEX                            */
    /* ================================================= */
```

```
counter = 0;

printf("Running WITHOUT mutex...\n");

/* Create threads */
for(i = 0; i < NUM_THREADS; i++)
{
   pthread_create(&threads[i], NULL,
                increment_without_mutex, NULL);
}

/* Wait for threads to finish */
for(i = 0; i < NUM_THREADS; i++)
{
   pthread_join(threads[i], NULL);
}

printf("Expected Counter Value = %d\n",
      NUM_THREADS * INCREMENTS);

printf("Actual Counter Value   = %lld\n", counter);

printf("\nRace condition occurs because multiple "
      "threads access and modify the shared "
      "counter simultaneously.\n");

/* ================================================= */
/* PART 2 : WITH MUTEX                               */
/* ================================================= */
```

```
counter = 0;

printf("\n----------------------------------------\n");
printf("Running WITH mutex...\n");

/* Initialize mutex */
pthread_mutex_init(&lock, NULL);

/* Create threads */
for(i = 0; i < NUM_THREADS; i++)
{
   pthread_create(&threads[i], NULL,
                increment_with_mutex, NULL);
}

/* Wait for threads to finish */
for(i = 0; i < NUM_THREADS; i++)
{
   pthread_join(threads[i], NULL);
}

printf("Expected Counter Value = %d\n",
      NUM_THREADS * INCREMENTS);

printf("Actual Counter Value   = %lld\n", counter);

printf("\nMutex solves the problem by allowing "
      "only one thread at a time to enter "
      "the critical section.\n");

/* Destroy mutex */
```

```
    pthread_mutex_destroy(&lock);


    return 0;
}
```